

Práctica Tema 2

Serafi Nebot Ginard

10/03/2025

Contents

1	Monitorización de la CPU	2
1.1	¿Cuántas CPUs tiene el sistema que se ha monitorizado? ¿De dónde se ha obtenido esa información?	2
1.2	¿Cuál es la utilización media de la CPU en modo usuario, sistema y en global?	2
1.3	¿Cómo se comportan las medidas anteriores a lo largo del tiempo de observación? Muestra las tres métricas de forma gráfica.	2
1.4	¿Cuál es la sobrecarga provocada por el monitor TOP?	3
2	Monitorización de la memoria principal	4
2.1	¿Qué capacidad total tiene la memoria principal del sistema? ¿De dónde se ha obtenido ese dato?	4
2.2	¿Cuál es la utilización media de la memoria? ¿Y la capacidad media utilizada?	4
2.3	¿Cómo se comporta la utilización de la memoria y la capacidad utilizada? Representa estas métricas gráficamente. .	4
2.4	¿Cuál es la sobrecarga provocada por el monitor VMSTAT?	6
3	Monitorización en paralelo	8
4	Pregunta voluntaria	11
4.1	Expresar la fórmula de la sobrecarga de la monitorización cuando el monitor es basado en ‘detección de eventos’. .	11

1 Monitorización de la CPU

El comando utilizado para la monitorización es el siguiente:

```
top -b -n 1080 -d 5 | awk '/%Cpu\s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", 100-$8 " ", $2 " ", $4}'
```

La utilización de CPU global se calcula con el propio comando ($U_{global} = 100 - U_{idle}$) y el output generado ya está en formato CSV, por lo que no necesita ningún post-procesado.

1.1 ¿Cuántas CPUs tiene el sistema que se ha monitorizado? ¿De dónde se ha obtenido esa información?

El sistema tiene 8 CPUs. Se ha obtenido con el siguiente comando:

```
$ nproc --all  
8
```

1.2 ¿Cuál es la utilización media de la CPU en modo usuario, sistema y en global?

- Media usuario: 0.073981 %
- Media sistema: 0.042222 %
- Media global: 0.155370 %

1.3 ¿Cómo se comportan las medidas anteriores a lo largo del tiempo de observación? Muestra las tres métricas de forma gráfica.

La utilización de la CPU se mantiene bastante baja, entre el 0% y 2%, y estable durante todo el período de medición, exceptuando el pequeño pico de actividad que hay entre las 09:50 y las 10:00. Las primeras mediciones son muy elevadas, probablemente debido a que el monitor top necesita un tiempo para estabilizar los valores medidos. En la gráfica se puede apreciar como la utilización global siempre es superior a la utilización de usuario y sistema.

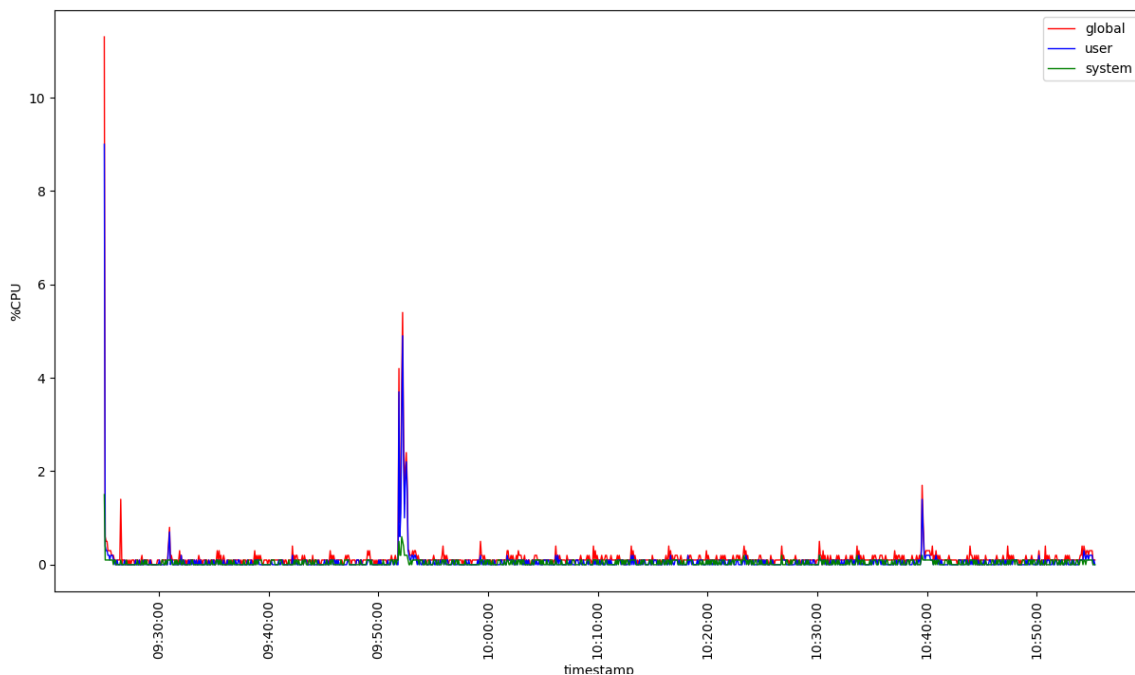


Figure 1: CPU 90 min

1.4 ¿Cuál es la sobrecarga provocada por el monitor TOP?

Ejecutando el siguiente unas varias veces podemos obtener el tiempo de ejecución de cada medición:

```
time top -b -n 1 -d 1 | awk '/%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
$ time top -b -n 1 -d 1 | awk '/^%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
2025-03-10 11:02:35,9.9,8.5,1.4
```

```
real    0m0.188s
user    0m0.010s
sys     0m0.027s
$ time top -b -n 1 -d 1 | awk '/^%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
2025-03-10 11:02:36,8.1,7.4,0.7
```

```
real    0m0.174s
user    0m0.013s
sys     0m0.012s
$ time top -b -n 1 -d 1 | awk '/^%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
2025-03-10 11:02:36,3.1,2.3,0.8
```

```
real    0m0.168s
user    0m0.006s
sys     0m0.013s
$ time top -b -n 1 -d 1 | awk '/^%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
2025-03-10 11:02:37,8.3,6.8,1.5
```

```
real    0m0.171s
user    0m0.006s
sys     0m0.016s
$ time top -b -n 1 -d 1 | awk '/^%Cpu\(s\):/ {print strftime("%Y-%m-%d %H:%M:%S") " ", " 100-$8 ", " $2 ", " $4}'
2025-03-10 11:02:37,6.2,5.4,0.8
```

```
real    0m0.170s
user    0m0.007s
sys     0m0.014s
```

Teniendo en cuenta varios resultados podemos calcular el tiempo medio de ejecución del monitor:

$$T_{monitor} = \frac{0.188 + 0.174 + 0.168 + 0.171 + 0.170}{5} = 0.174 \text{ s}$$

Esto significa que la sobrecarga del monitor es:

$$\frac{T_{monitor}}{T_{medida}} = \frac{0.174}{5} = 0.0348 = 3.48\%$$

2 Monitorización de la memoria principal

El comando utilizado para la monitorización es el siguiente:

```
vmstat 3 3600 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
```

Este comando sólo proporciona el timestamp y la utilización de RAM actual. Debido a que los otros dos campos se pueden calcular a partir de la utilización y de la capacidad total, que es una constante, he decidido hacer estos cálculos con un script de Python que genera el CSV final. Además, el comando genera una cabecera cada cierto tiempo que el script también se encarga de quitarla.

2.1 ¿Qué capacidad total tiene la memoria principal del sistema? ¿De dónde se ha obtenido ese dato?

La capacidad de memoria principal del sistema es de 12520767488 B = 11.66 GB. Se ha obtenido con el siguiente comando:

```
$ free --bytes
```

	total	used	free	shared	buff/cache	available
Mem:	12520767488	923324416	10237382656	42438656	1360060416	11346395136
Swap:	4294963200	0	4294963200			

2.2 ¿Cuál es la utilización media de la memoria? ¿Y la capacidad media utilizada?

- Capacidad media: 8427075.82 KB
- Utilización media: 3800236.17 KB; representada en porcentaje:

$$\frac{U_{media}}{C_{total}} \cdot 100 = \frac{3800236.17}{\frac{12520767488}{2^{10}}} \cdot 100 = 31.07\%$$

2.3 ¿Cómo se comporta la utilización de la memoria y la capacidad utilizada? Representa estas métricas gráficamente.

La utilización de la memoria experimenta cambios muy rápidos, probablemente debido al encendido/apagado de programas. Como se aprecia en la gráfica, y es obvio por definición, la cantidad de memoria libre y usada tienen un comportamiento inverso, generando un efecto espejo sobre el eje horizontal.

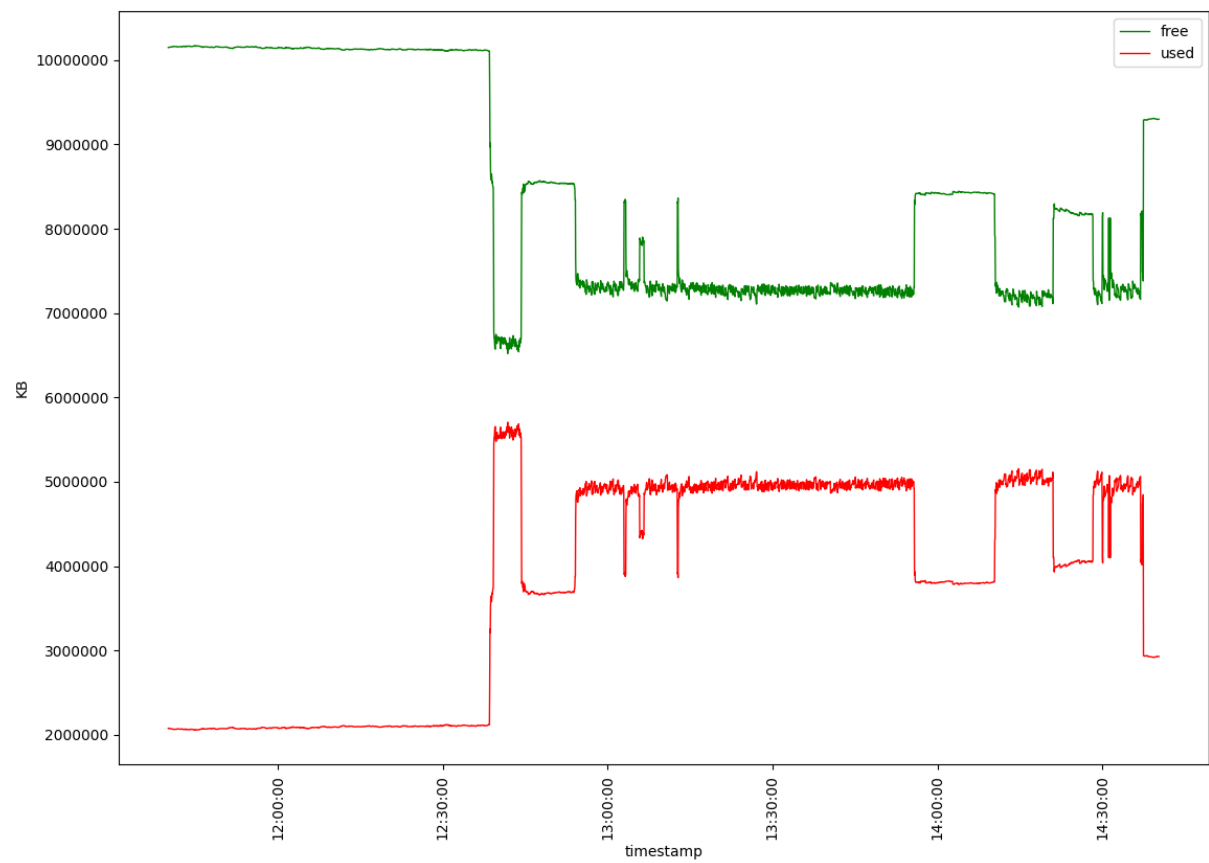


Figure 2: RAM 2h absoluto

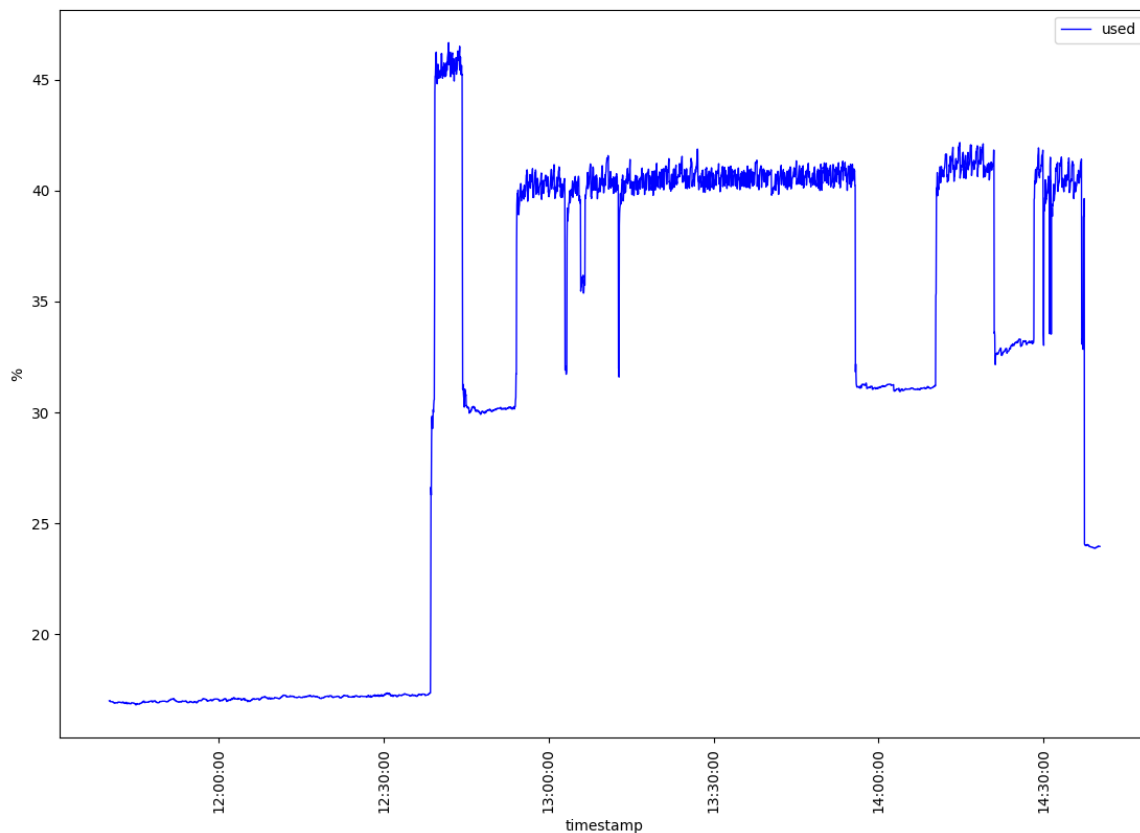


Figure 3: RAM 2h porcentaje

2.4 ¿Cuál es la sobrecarga provocada por el monitor VMSTAT?

Ejecutando el siguiente comando unas varias veces podemos obtener el tiempo de ejecución de cada uno:

```
time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
$ time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
2025-03-09 19:57:39 -----io-----
2025-03-09 19:57:39 free
2025-03-09 19:57:39 10580112

real    0m0.002s
user    0m0.004s
sys     0m0.000s
$ time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
2025-03-09 19:57:41 -----io-----
2025-03-09 19:57:41 free
2025-03-09 19:57:41 10580112

real    0m0.002s
user    0m0.000s
sys     0m0.003s
$ time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
2025-03-09 19:57:41 -----io-----
2025-03-09 19:57:41 free
2025-03-09 19:57:41 10580364
```

```

real    0m0.006s
user    0m0.003s
sys     0m0.000s
$ time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
2025-03-09 19:57:42 -----io-----
2025-03-09 19:57:42 free
2025-03-09 19:57:42 10580364

```

```

real    0m0.001s
user    0m0.001s
sys     0m0.001s
$ time vmstat 1 1 --unit K --wide | awk '{print strftime("%Y-%m-%d %H:%M:%S"), $4}'
2025-03-09 19:57:43 -----io-----
2025-03-09 19:57:43 free
2025-03-09 19:57:43 10580112

```

```

real    0m0.002s
user    0m0.003s
sys     0m0.000s

```

Teniendo en cuenta varios resultados podemos calcular el tiempo medio de ejecución del monitor:

$$T_{monitor} = \frac{0.002 + 0.002 + 0.006 + 0.001 + 0.002}{5} = 0.0026 \text{ s}$$

Esto significa que la sobrecarga del monitor es:

$$\frac{T_{monitor}}{T_{medida}} = \frac{0.0026}{3} = 0.00086 = 0.086\%$$

3 Monitorización en paralelo

En los anteriores apartados hemos utilizado `top` y `vmstat` para monitorizar la CPU y la RAM en modo *batch*. Debido a que necesitan un tiempo de estabilización (los primeros valores de ambos monitores no son fiables), no podemos utilizarlos en conjunto en nuestro script bash porque por cada iteración solo vamos a obtener el primer valor de cada monitor.

Para la CPU, he decidido utilizar la primera línea del archivo `/proc/stat` que indica el acumulado de utilización de la CPU en el siguiente formato (las unidades son *jiffies*, pero debido a que queremos un porcentaje esto es irrelevante):

```
cpu  user nice system idle iowait irq softirq steal guest guest_nice
```

Para obtener la utilización de cada campo durante el intervalo de tiempo que indiquemos, vamos a necesitar tomar una medida, esperar el intervalo de tiempo de monitorización, tomar otra medida y calcular la diferencia.

Si la utilización total, U_t , es la suma de todos los campos y U_i es el campo idle, el porcentaje de utilización, U , es:

$$U = \frac{U_t - U_i}{U_t} \cdot 100$$

Para la RAM, he decidido utilizar el comando `free --bytes` que genera el siguiente resultado:

```
$ free --bytes
              total          used          free          shared  buff/cache  available
Mem:      12520767488    934035456 10130608128    43511808   1456123904 11316568064
Swap:      4294963200              0    4294963200
```

de donde podemos obtener la utilización actual:

```
$ free --bytes | sed -n '2p' | awk '{print $3}'
930934784
```

y calcular el porcentaje de utilización, $\frac{used}{total} \cdot 100$:

```
$ free --bytes | sed -n '2p' | awk '{print $3/$2*100}'
7.41111
```

He juntado estos dos monitores en un script bash para que se ejecuten cada 5 segundos, teniendo en cuenta el tiempo de ejecución de ambos monitores tomando el *epoch* en milisegundos antes y después de su ejecución con `date +%s%3N` para compensar el tiempo de monitorización y poder calcular el *overhead* y decidir el intervalo de monitorización:

```
#!/bin/bash
```

```
prev_line=$(grep 'cpu ' /proc/stat)
```

```
# time_start and time_end are used to calculate the amount of time it takes to execute the monitor
time_start=$(date +%s%3N)
```

```
# loop 1440 times (2 hours / 5 seconds per iteration = 7200 / 5 = 1440)
```

```
for i in $(seq 1 1440); do
    # calculate the time to sleep taking into account monitor execution time
    time_end=$(date +%s%3N)
    time_elapsed=$((time_end - time_start))
    # echo $time_elapsed ms
    time_sleep=$(echo "scale=3; 5 - ($time_elapsed) / 1000" | bc)
    sleep $time_sleep
```

```
time_start=$(date +%s%3N)
```

```
current_line=$(grep 'cpu ' /proc/stat)
```

```
# sum all fields from user time onward (skip "cpu" label at $1)
```

```
prev_total=$(echo "$prev_line" | awk '{sum=0; for(i=2; i<=NF; i++) sum+=i; print sum}')
```

```
# idle time is the 4th field after "cpu" (field $5)
```

```
prev_idle=$(echo "$prev_line" | awk '{print $5}')
```



```

# calculate total cpu time for current reading
current_total=$(echo "$current_line" | awk '{sum=0; for(i=2; i<=NF; i++) sum+=$i; print sum}')
current_idle=$(echo "$current_line" | awk '{print $5}')

# compute differences between current and previous readings
diff_total=$((current_total - prev_total))
diff_idle=$((current_idle - prev_idle))

# calculate cpu usage percentage
# usage = ((total - idle) / total) * 100
if [ "$diff_total" -gt 0 ]; then
    cpu_usage=$(echo "scale=2; ($diff_total - $diff_idle) * 100 / $diff_total" | bc)
else
    cpu_usage=0
fi

# get ram usage in bytes from free command
ram_line=$(free --bytes | sed -n '2p')
ram_abs=$(echo $ram_line | awk '{print $3}')
ram_percent=$(echo $ram_line | awk '{print $3/$2*100}')

# print timestamp, cpu and ram usage
printf "%s,%.4f,%.4f\n" "$(date '+%Y-%m-%d %H:%M:%S')" $cpu_usage $ram_abs $ram_percent

# update previous reading for the next iteration
prev_line="$current_line"
done

```

Descomentando la línea que imprime el tiempo que tarda el monitor en milisegundos podemos obtener varias medidas para calcular la media y después calcular el overhead:

```

3 ms
2025-03-08 12:54:26,1.1500,925650944,7.3929
21 ms
2025-03-08 12:54:31,0.8500,917913600,7.3311
18 ms
2025-03-08 12:54:36,0.9400,918171648,7.3332
22 ms
2025-03-08 12:54:40,0.9200,918683648,7.3373
19 ms
2025-03-08 12:54:45,0.8700,919191552,7.3413
18 ms
2025-03-08 12:54:50,0.9200,918933504,7.3393
21 ms

```

Obviando la primera medida porque aún no se ha ejecutado el monitor, obtenemos una media de:

$$T_{monitor} = \frac{21 + 18 + 22 + 19 + 18 + 21}{6} = 19.833 \text{ ms}$$

Para escoger un intervalo de monitorización, $T_{intervalo}$, tal que el overhead sea menor que 10%:

$$\frac{T_{monitor}}{T_{intervalo}} \cdot 100 < 10 \implies \frac{T_{monitor}}{10} \cdot 100 < T_{intervalo}$$

$$T_{intervalo} > \frac{19.833}{10} \cdot 100 = T_{intervalo} > 198.33 \text{ ms}$$

He escogido un $T_{intervalo}$ de 5000 ms, lo que implica un overhead de $\frac{19.833}{5000} \cdot 100 = 0.39\%$, muy por debajo del límite de 10%.

En la gráfica se puede apreciar que todas las variables siguen, mas o menos, el mismo patrón con cambios muy rápidos debido a la apertura y cierre de aplicaciones, pero cuando no hay ninguna carga de trabajo importante se mantienen a unos valores bastante bajos y estables:

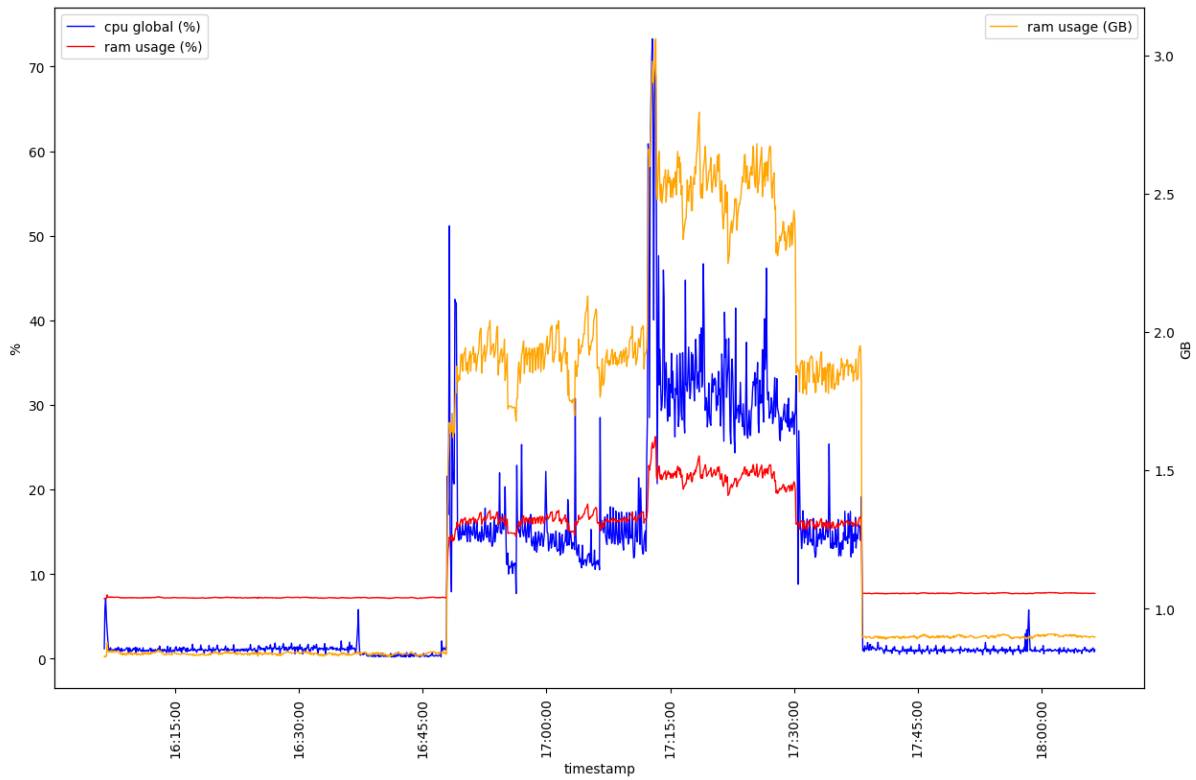


Figure 4: CPU & RAM

4 Pregunta voluntaria

4.1 Expresar la fórmula de la sobrecarga de la monitorización cuando el monitor es basado en 'detección de eventos'.

Para un intervalo de monitorización constante el overhead se puede calcular teniendo en cuenta sólo un intervalo de tiempo o la suma de todo el conjunto. Si T_M es el tiempo de monitorización, T_E el tiempo de ejecución y T_I el intervalo de tiempo y n el número de medidas, el overhead, O , es el siguiente:

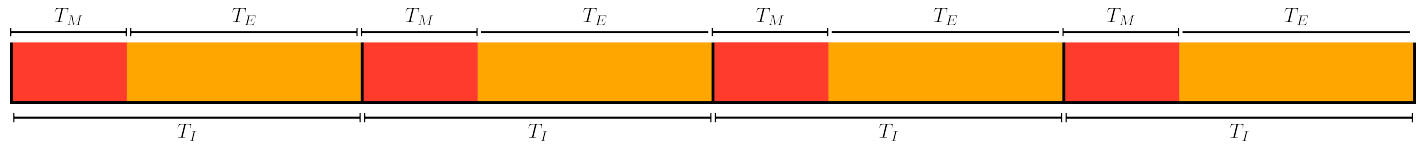


Figure 5: Intervalo monitorización constante

$$O = \frac{T_M \cdot n}{T_I \cdot n} = \frac{T_M}{T_I}$$

Para un intervalo de monitorización variable (dirigido por eventos) se puede obtener el overhead calculando tiempo total de monitorización y el tiempo total de los intervalos entre eventos:

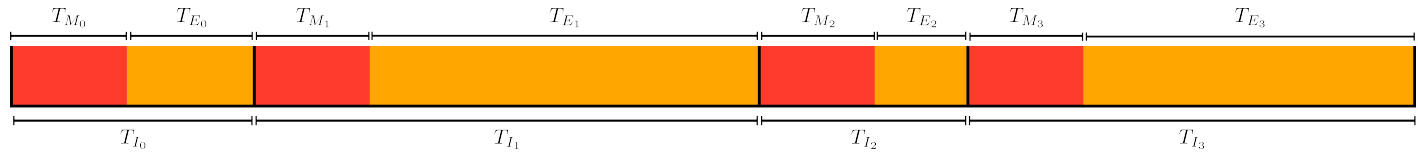


Figure 6: Intervalo monitorización variable

$$O = \frac{\sum_{i=0}^n T_{M_i}}{\sum_{i=0}^n T_{I_i}}$$